```
from __future__ import annotations

import logging
import os
import re
import traceback
from copy import deepcopy
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Dict, List, Optional

from app.services.csd_refresh_service import CSDRefreshService
from app.services.monthly_quotes_storage import
MonthlyQuotesStorageService
from app.services.quote_engine import build_quote_from_rfq
from app.services.quote_pack_service import generate_quote_pack
from app.services.supplier_award_execution_service import
execute_supplier_award_workflow
from app.services.supplier_quote_ingestion_service import
ingest_supplier_quotes
from app.services.supplier_rfq_request_service import
SupplierRFQRequestService
from app.services.supply_classifier import classify_supply_rfq

logger = logging.getLogger(__name__)


class TenderPipelineService:
    MIN_PROFIT_AMOUNT = 30000.0
    DEFAULT_MARGIN_FLOOR = 0.25
    DEFAULT_TEST_UNIT_PRICE = 1000.00
    DEFAULT_VAT_RATE = 0.15

    DISALLOWED_CATEGORY_KEYWORDS = [
        "medical consumables",
        "medical equipment",
        "pharmaceutical",
        "pharmaceuticals",
        "surgical",
        "it equipment",
        "information technology",
        "computer equipment",
        "laptop",
        "laptops",
        "printer",
        "printers",
        "server",
        "servers",
        "petrol",
        "diesel",
        "fuel",
        "construction",
        "building",
        "repair",
        "maintenance",
        "civil works",
        "installation",
        "renovation",
```

```
        "upgrading",
        "refurbishment",
        "plumbing works",
        "electrical works",
        "catering",
        "food",
        "meals",
        "Hospitality",
        "catering services",
        "refreshments",
        "kitchen services",
    ]

    SUPPLY_KEYWORDS = [
        "supply",
        "supply and delivery",
        "delivery",
        "deliver",
        "procurement of",
        "purchase of",
        "supply, delivery",
        "supply and install",
        "goods",
        "materials",
        "equipment",
        "ppe",
        "consumables",
    ]

    PHYSICAL_METHODS = {
        "physical",
        "courier",
        "hand",
        "hand delivery",
        "hand-delivery",
        "manual",
    }

    PORTAL_METHODS = {
        "portal",
        "etender",
        "e-tender",
        "eprocurement",
        "e-procurement",
        "online",
        "website",
    }

    EMAIL_METHODS = {
        "email",
        "e-mail",
        "mail",
    }

    RFQ_REGEX_PATTERNS = [
        r"\b(?:RFQ|RFP|BID|TENDER|TNDR|QUOTATION|QUOTE)\s*[:#-]?\s*([A-
Z0-9][A-Z0-9/\-_.]{2,})\b",
        r"\b([A-Z]{2,}[-/][A-Z0-9][A-Z0-9/\-_.]{2,})\b",
```

```
        r"\b([A-Z0-9]{2,}[-/][A-Z0-9][A-Z0-9/\-_.]{2,})\b",
    ]

    @staticmethod
    def _now_iso() -> str:
        return datetime.now(timezone.utc).isoformat()

    @classmethod
    def _log_stage(cls, stage: str, payload: Optional[Dict[str, Any]] =
None) -> None:
        try:
            buyer_rfq_number = ""
            quote_number = ""
            if isinstance(payload, dict):
                buyer_rfq_number = cls._safe_str(
                    payload.get("_locked_buyer_rfq_number")
                    or payload.get("buyer_rfq_number")
                    or payload.get("rfq_number")
                    or payload.get("reference_number")
                    or payload.get("document_number")
                )
                quote_number = cls._safe_str(payload.get("quote_number"))
            logger.info(
                "[PIPELINE] %s | buyer_rfq_number=%s | quote_number=%s",
                stage,
                buyer_rfq_number or "-",
                quote_number or "-",
            )
            print(
                f"[PIPELINE] {stage} | "
                f"buyer_rfq_number={buyer_rfq_number or '-'} | "
                f"quote_number={quote_number or '-'}"
            )
        except Exception:
            pass

    @classmethod
    def _safe_str(cls, value: Any, default: str = "") -> str:
        if value is None:
            return default
        text = str(value).strip()
        return text if text else default

    @classmethod
    def _to_float(cls, value: Any, default: float = 0.0) -> float:
        try:
            if value is None or value == "":
                return default
            cleaned = (
                str(value)
                .replace(",", "")
                .replace("R", "")
                .replace("ZAR", "")
                .replace("zar", "")
                .strip()
            )
            return float(cleaned)
        except Exception:
```

```
            return default

    @classmethod
    def _to_bool(cls, value: Any, default: bool = False) -> bool:
        if isinstance(value, bool):
            return value
        if value is None:
            return default
        if isinstance(value, (int, float)):
            return bool(value)

        text = cls._safe_str(value).lower()
        if text in {"true", "1", "yes", "y", "on"}:
            return True
        if text in {"false", "0", "no", "n", "off"}:
            return False
        return default

    @classmethod
    def _pick_first_non_empty(cls, *values: Any, default: str = "") ->
str:
        for value in values:
            text = cls._safe_str(value)
            if text:
                return text
        return default

    @classmethod
    def _safe_dict(cls, value: Any) -> Dict[str, Any]:
        return deepcopy(value) if isinstance(value, dict) else {}

    @classmethod
    def _dedupe_string_list(cls, values: List[Any]) -> List[str]:
        deduped: List[str] = []
        seen = set()
        for value in values:
            text = cls._safe_str(value)
            if not text:
                continue
            key = text.lower()
            if key in seen:
                continue
            seen.add(key)
            deduped.append(text)
        return deduped

    @classmethod
    def _should_skip_external_calls(cls, payload: Dict[str, Any]) ->
bool:
        return cls._to_bool(payload.get("skip_external_calls"), False)

    @classmethod
    def _should_skip_email_submission(cls, payload: Dict[str, Any]) ->
bool:
        return cls._to_bool(payload.get("skip_email_submission"), False)

    @classmethod
    def _is_unknown_token(cls, value: Any) -> bool:
```

```
        text = cls._safe_str(value).strip().lower()
        return text in {
            "",
            "unknown",
            "unknown-rfq",
            "rfq-unknown",
            "n/a",
            "na",
            "-",
            "none",
            "null",
            "buyer / issuing entity",
            "supply and delivery item",
            "client",
            "lmcp-quote",
        }

    @classmethod
    def _is_meaningful_text(cls, value: Any) -> bool:
        return not cls._is_unknown_token(value)

    @classmethod
    def _normalize_text_blob(cls, rfq: Dict[str, Any]) -> str:
        parts = [
            cls._safe_str(rfq.get("title")),
            cls._safe_str(rfq.get("description")),
            cls._safe_str(rfq.get("scope")),
            cls._safe_str(rfq.get("category")),
            cls._safe_str(rfq.get("buyer_name")),
            cls._safe_str(rfq.get("department")),
            cls._safe_str(rfq.get("tender_number")),
            cls._safe_str(rfq.get("buyer_rfq_number")),
            cls._safe_str(rfq.get("rfq_number")),
            cls._safe_str(rfq.get("reference_number")),
            cls._safe_str(rfq.get("document_number")),
            cls._safe_str(rfq.get("entity_name")),
            cls._safe_str(rfq.get("entity_type")),
            cls._safe_str(rfq.get("source_type")),
            cls._safe_str(rfq.get("submission_method")),
            cls._safe_str(rfq.get("subject")),
            cls._safe_str(rfq.get("email_subject")),
        ]
        return " | ".join([p.lower() for p in parts if p])

    @classmethod
    def _normalize_submission_method(cls, rfq: Dict[str, Any]) -> str:
        submission_pack = cls._safe_dict(rfq.get("submission_pack"))

        raw = cls._pick_first_non_empty(
            rfq.get("submission_method"),
            submission_pack.get("submission_method"),
            rfq.get("delivery_method"),
            rfq.get("bid_submission_method"),
            rfq.get("submission_mode"),
            default="unknown",
        )
        method = cls._safe_str(raw).lower()
```

```
        if method in cls.EMAIL_METHODS:
            return "email"
        if method in cls.PORTAL_METHODS:
            return "portal"
        if method in cls.PHYSICAL_METHODS:
            return "physical"

        merged = cls._normalize_text_blob(rfq)
        if "email" in merged or "e-mail" in merged:
            return "email"
        if "portal" in merged or "etender" in merged or "e-tender" in
merged:
            return "portal"
        if "hand delivery" in merged or "courier" in merged or "tender
box" in merged:
            return "physical"

        return "unknown"

    @classmethod
    def _normalize_recipient_email(cls, rfq: Dict[str, Any]) ->
Optional[str]:
        submission_pack = cls._safe_dict(rfq.get("submission_pack"))
        buyer = cls._safe_dict(rfq.get("buyer"))
        source_rfq = cls._safe_dict(rfq.get("source_rfq"))

        candidates = [
            rfq.get("recipient_email"),
            rfq.get("submission_email"),
            rfq.get("contact_email"),
            rfq.get("buyer_email"),
            rfq.get("email"),
            submission_pack.get("recipient_email"),
            buyer.get("email"),
            source_rfq.get("submission_email"),
            source_rfq.get("recipient_email"),
            source_rfq.get("buyer_email"),
        ]
        for value in candidates:
            text = cls._safe_str(value)
            if "@" in text:
                return text
        return None

    @classmethod
    def _has_briefing_session(cls, rfq: Dict[str, Any]) -> bool:
        explicit = rfq.get("briefing_required")
        if explicit is True:
            return True

        text = cls._normalize_text_blob(rfq)
        briefing_markers = [
            "briefing session",
            "compulsory briefing",
            "mandatory briefing",
            "compulsory site inspection",
            "mandatory site inspection",
            "site briefing",
```

```
            "briefing meeting",
        ]
        return any(marker in text for marker in briefing_markers)

    @classmethod
    def _looks_like_supply_tender(cls, rfq: Dict[str, Any]) -> bool:
        text = cls._normalize_text_blob(rfq)

        if any(bad in text for bad in cls.DISALLOWED_CATEGORY_KEYWORDS):
            return False

        has_supply_signal = any(word in text for word in
cls.SUPPLY_KEYWORDS)
        has_works_signal = any(word in text for word in
cls.WORKS_KEYWORDS)

        if has_works_signal and not has_supply_signal:
            return False

        category = cls._safe_str(rfq.get("category")).lower()
        if "supply" in category or "goods" in category or "delivery" in
category:
            return True

        return has_supply_signal

    @classmethod
    def _extract_line_items(cls, rfq: Dict[str, Any]) -> List[Dict[str,
Any]]:
        for key in (
            "line_items",
            "items",
            "pricing_schedule_items",
            "buyer_pricing_schedule",
            "pricing_schedule",
        ):
            items = rfq.get(key)
            if isinstance(items, list) and items:
                return [deepcopy(x) for x in items if isinstance(x,
dict)]
        return []

    @classmethod
    def _estimate_financials(cls, rfq: Dict[str, Any]) -> Dict[str, Any]:
        totals = cls._safe_dict(rfq.get("totals"))

        estimated_revenue = cls._to_float(
            rfq.get("estimated_revenue")
            or rfq.get("quote_total")
            or rfq.get("total_quote_amount")
            or rfq.get("estimated_value")
            or rfq.get("budget")
            or totals.get("subtotal_excl_vat")
            or totals.get("total_incl_vat")
        )

        estimated_cost = cls._to_float(
            rfq.get("estimated_cost")
```

```
            or rfq.get("cost_estimate")
            or rfq.get("supplier_cost_total")
            or rfq.get("selected_quote_total")
        )

        if estimated_revenue > 0 and estimated_cost <= 0:
            estimated_cost = estimated_revenue * (1.0 -
cls.DEFAULT_MARGIN_FLOOR)

        if estimated_cost > 0 and estimated_revenue <= 0:
            estimated_revenue = estimated_cost / max(0.01, (1.0 -
cls.DEFAULT_MARGIN_FLOOR))

        profit = max(0.0, estimated_revenue - estimated_cost) if
estimated_revenue > 0 else 0.0
        margin = (profit / estimated_revenue) if estimated_revenue > 0
else 0.0

        return {
            "estimated_revenue": round(estimated_revenue, 2),
            "estimated_cost": round(estimated_cost, 2),
            "estimated_profit": round(profit, 2),
            "estimated_margin": round(margin, 4),
        }

    @classmethod
    def _extract_rfq_from_text(cls, *texts: Any) -> str:
        for text in texts:
            blob = cls._safe_str(text)
            if not blob:
                continue
            for pattern in cls.RFQ_REGEX_PATTERNS:
                match = re.search(pattern, blob, flags=re.IGNORECASE)
                if match:
                    candidate = cls._safe_str(match.group(1))
                    if candidate and not
cls._is_unknown_token(candidate):
                        return candidate
        return ""

    @classmethod
    def _normalize_rfq_number(cls, payload: Dict[str, Any]) -> str:
        if not isinstance(payload, dict):
            return ""

        buyer = cls._safe_dict(payload.get("buyer"))
        rfq = cls._safe_dict(payload.get("rfq"))
        opportunity = cls._safe_dict(payload.get("opportunity"))
        source_rfq = cls._safe_dict(payload.get("source_rfq"))
        submission_pack = cls._safe_dict(payload.get("submission_pack"))
        buyer_schedule =
cls._safe_dict(payload.get("buyer_pricing_schedule"))
        pricing_schedule_mapped =
cls._safe_dict(payload.get("pricing_schedule_mapped"))
        original = cls._safe_dict(payload.get("_original_input_payload"))

        candidates = [
            payload.get("_locked_buyer_rfq_number"),
```

```
        payload.get("buyer_rfq_number"),
        payload.get("reference_number"),
        payload.get("rfq_number"),
        payload.get("tender_number"),
        payload.get("document_number"),
        payload.get("bid_number"),
        payload.get("notice_number"),
        payload.get("opportunity_number"),
        submission_pack.get("buyer_rfq_number"),
        submission_pack.get("document_number"),
        rfq.get("buyer_rfq_number"),
        rfq.get("rfq_number"),
        rfq.get("tender_number"),
        rfq.get("reference_number"),
        rfq.get("document_number"),
        rfq.get("bid_number"),
        rfq.get("notice_number"),
        opportunity.get("buyer_rfq_number"),
        opportunity.get("rfq_number"),
        opportunity.get("reference_number"),
        opportunity.get("document_number"),
        source_rfq.get("buyer_rfq_number"),
        source_rfq.get("rfq_number"),
        source_rfq.get("tender_number"),
        source_rfq.get("reference_number"),
        source_rfq.get("document_number"),
        buyer.get("rfq_number"),
        buyer_schedule.get("rfq_number"),
        buyer_schedule.get("reference_number"),
        pricing_schedule_mapped.get("rfq_number"),
        pricing_schedule_mapped.get("reference_number"),
        original.get("_locked_buyer_rfq_number"),
        original.get("buyer_rfq_number"),
        original.get("reference_number"),
        original.get("rfq_number"),
        original.get("tender_number"),
        original.get("document_number"),
    ]

    for candidate in candidates:
        text = cls._safe_str(candidate)
        if text and not cls._is_unknown_token(text):
            return text

    extracted = cls._extract_rfq_from_text(
        payload.get("subject"),
        payload.get("email_subject"),
        payload.get("title"),
        payload.get("description"),
        payload.get("quote_subject"),
        rfq.get("title"),
        rfq.get("description"),
        opportunity.get("title"),
        opportunity.get("description"),
        source_rfq.get("title"),
        source_rfq.get("description"),
        original.get("subject"),
        original.get("email_subject"),
```

```
            original.get("title"),
            original.get("description"),
        )
        if extracted:
            return extracted

        return ""

    @classmethod
    def _lock_original_payload(cls, payload: Dict[str, Any]) -> Dict[str,
Any]:
        result = deepcopy(payload if isinstance(payload, dict) else {})
        current_original = result.get("_original_input_payload")
        if isinstance(current_original, dict) and current_original:
            return result
        result["_original_input_payload"] = deepcopy(result)
        return result

    @classmethod
    def _force_rfq_into_payload(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        result = deepcopy(payload if isinstance(payload, dict) else {})

        rfq_number = cls._normalize_rfq_number(result)

        locked_buyer_rfq_number = cls._safe_str(
            result.get("_locked_buyer_rfq_number")
            or result.get("buyer_rfq_number")
            or rfq_number
        )

        if not locked_buyer_rfq_number or
cls._is_unknown_token(locked_buyer_rfq_number):
            locked_buyer_rfq_number = f"RFQ-
{datetime.utcnow().strftime('%Y%m%d%H%M%S')}"

        result["_locked_buyer_rfq_number"] = locked_buyer_rfq_number
        result["buyer_rfq_number"] = locked_buyer_rfq_number
        result["rfq_number"] = locked_buyer_rfq_number
        result["reference_number"] = locked_buyer_rfq_number
        result["document_number"] = locked_buyer_rfq_number

        submission_pack = cls._safe_dict(result.get("submission_pack"))
        submission_pack["buyer_rfq_number"] = locked_buyer_rfq_number
        if not cls._safe_str(submission_pack.get("document_number")) or
cls._is_unknown_token(
            submission_pack.get("document_number")
        ):
            submission_pack["document_number"] = locked_buyer_rfq_number
        result["submission_pack"] = submission_pack

        buyer = cls._safe_dict(result.get("buyer"))
        if not cls._safe_str(buyer.get("rfq_number")) or
cls._is_unknown_token(buyer.get("rfq_number")):
            buyer["rfq_number"] = locked_buyer_rfq_number
        result["buyer"] = buyer

        return result
```

```
    @classmethod
    def _normalize_rfq(cls, rfq: Dict[str, Any]) -> Dict[str, Any]:
        item = deepcopy(rfq if isinstance(rfq, dict) else {})
        item = cls._lock_original_payload(item)

        if not isinstance(item.get("source_rfq"), dict):
            item["source_rfq"] = deepcopy(item)

        source_rfq = cls._safe_dict(item.get("source_rfq"))
        buyer = cls._safe_dict(item.get("buyer"))

        item["pipeline_processed_at"] = cls._now_iso()
        item["title"] = cls._pick_first_non_empty(
            item.get("title"),
            item.get("description"),
            source_rfq.get("title"),
            source_rfq.get("description"),
            default="Supply and delivery item",
        )
        item["description"] = cls._pick_first_non_empty(
            item.get("description"),
            item.get("title"),
            default=item["title"],
        )
        item["buyer_name"] = cls._pick_first_non_empty(
            item.get("buyer_name"),
            buyer.get("company_name"),
            buyer.get("name"),
            source_rfq.get("buyer_name"),
            default="Buyer / Issuing Entity",
        )
        item["submission_method"] =
cls._normalize_submission_method(item)
        item["recipient_email"] = cls._normalize_recipient_email(item)
        item["line_items"] = cls._extract_line_items(item)

        item = cls._force_rfq_into_payload(item)

        financials = cls._estimate_financials(item)
        item.update(financials)
        return item

    @classmethod
    def _classify_rfq(cls, rfq: Dict[str, Any]) -> Dict[str, Any]:
        try:
            result = classify_supply_rfq(rfq)
            if isinstance(result, dict):
                return result
        except Exception:
            logger.exception("classify_supply_rfq failed, using fallback
classification.")

    @classmethod
    def _classify_rfq(cls, rfq: Dict[str, Any]) -> Dict[str, Any]:
        text = cls._normalize_text_blob(rfq)

        catering_markers = [
```

```
            "catering",
            "catering services",
            "food",
            "foods",
            "meal",
            "meals",
            "refreshment",
            "refreshments",
            "beverage",
            "beverages",
            "hospitality",
            "kitchen services",
        ]

        # □üîí HARD BLOCK ,Äî BEFORE ANYTHING ELSE
        if any(marker in text for marker in catering_markers):
            return {
                "is_supply": False,
                "briefing_required": False,
                "profitable": False,
                "eligible": False,
                "quote_ready": False,
                "classification_reasons": ["Catering tender blocked by
policy"],
            }

        # Try external classifier AFTER block
        try:
            result = classify_supply_rfq(rfq)
            if isinstance(result, dict):
                result_text = cls._normalize_text_blob(result)

                # □üîí Double protection (in case classifier overrides)
                if any(marker in text or marker in result_text for marker
in catering_markers):
                    result["is_supply"] = False
                    result["eligible"] = False
                    result["quote_ready"] = False

                    reasons = result.get("classification_reasons") or []
                    if not isinstance(reasons, list):
                        reasons = [str(reasons)]
                    reasons.append("Catering tender blocked by policy")
                    result["classification_reasons"] = reasons

                return result

        except Exception:
            logger.exception("classify_supply_rfq failed, using fallback
classification.")

        # Fallback logic
        return {
            "is_supply": is_supply,
            "briefing_required": briefing_required,
            "profitable": profitable,
            "eligible": eligible,
            "quote_ready": quote_ready,
```

```
            "classification_reasons": reasons,
        }


        eligible = bool(is_supply and not briefing_required and
profitable)
        quote_ready = eligible

        return {
            "is_supply": is_supply,
            "briefing_required": briefing_required,
            "profitable": profitable,
            "eligible": eligible,
            "quote_ready": quote_ready,
            "classification_reasons": reasons,
        }

        is_supply = cls._looks_like_supply_tender(rfq)
        briefing_required = cls._has_briefing_session(rfq)
        profitable = False
        reasons: List[str] = []

        is_supply = cls._looks_like_supply_tender(rfq)
        briefing_required = cls._has_briefing_session(rfq)
        profitable = False
        reasons: List[str] = []

        if not is_supply:
            reasons.append("Not supply-and-delivery")
        if briefing_required:
            reasons.append("Briefing session required")

        if is_supply and not briefing_required:
            financials = cls._estimate_financials(rfq)
            profitable = (
                financials["estimated_profit"] >= cls.MIN_PROFIT_AMOUNT
                and financials["estimated_margin"] >=
cls.DEFAULT_MARGIN_FLOOR
            )
            if not profitable:
                reasons.append("Below minimum profit/margin threshold")

        eligible = is_supply and not briefing_required and profitable
        return {
            "is_supply": is_supply,
            "briefing_required": briefing_required,
            "profitable": profitable,
            "eligible": eligible,
            "quote_ready": eligible,
            "classification_reasons": reasons,
        }

    @classmethod
    def _finalize_classification_flags(cls, result: Dict[str, Any]) ->
Dict[str, Any]:
        payload = deepcopy(result)
        payload["eligible"] = cls._to_bool(payload.get("eligible"),
False)
```

```
        payload["quote_ready"] = cls._to_bool(payload.get("quote_ready"),
payload["eligible"])

        force_pipeline = cls._to_bool(
            payload.get("force_quote_ready")
            or payload.get("force_pipeline")
            or payload.get("pipeline_test_mode"),
            False,
        )

        if force_pipeline:
            payload["eligible"] = True
            payload["quote_ready"] = True
            payload["classification_reasons"] =
payload.get("classification_reasons") or [
                "Forced quote-ready for pipeline testing"
            ]

        # ================================
        # □üîí FORCE RECIPIENT EMAIL (CRITICAL FIX)
        # ================================
        if payload.get("submission_method") == "email":
            payload["recipient_email"] = (
                payload.get("recipient_email")
                or payload.get("buyer_email")
                or payload.get("buyer", {}).get("email")
                or "lmcpaqsystem@gmail.com"
            )

        elif payload.get("submission_method") == "physical":
            payload["recipient_email"] = "lmcpaqsystem@gmail.com"

        # ================================
        # □üîí FORCE SUBMISSION CHANNEL
        # ================================
        submission_method = str(payload.get("submission_method") or
"").lower().strip()

        if submission_method == "email":
            payload["submission_method"] = "email"
            payload["submission_channel"] = "email"
            payload["recipient_email"] = (
                payload.get("recipient_email")
                or payload.get("buyer_email")
                or payload.get("buyer", {}).get("email")
                or "lmcpaqsystem@gmail.com"
            )

        elif submission_method == "portal":
            payload["submission_method"] = "portal"
            payload["submission_channel"] = "portal"

        elif submission_method == "physical":
            payload["submission_method"] = "physical"
            payload["submission_channel"] = "physical_via_email"
            payload["recipient_email"] = "lmcpaqsystem@gmail.com"

        else:
```

```
            payload["submission_method"] = "email"
            payload["submission_channel"] = "email"
            payload["recipient_email"] = "lmcpaqsystem@gmail.com"

        # ================================
        # □üîí FINAL SUBMISSION LOCK (LAST LINE DEFENSE)
        # ================================
        submission_method = payload.get("submission_method")

        if submission_method == "email":
            payload["submission_channel"] = "email"
            payload["recipient_email"] = (
                payload.get("recipient_email")
                or payload.get("buyer_email")
                or payload.get("buyer", {}).get("email")
                or "lmcpaqsystem@gmail.com"
            )

        elif submission_method == "portal":
            payload["submission_channel"] = "portal"

        elif submission_method == "physical":
            payload["submission_channel"] = "physical_via_email"
            payload["recipient_email"] = "lmcpaqsystem@gmail.com"

        # Never allow empty email if email-based submission
        if payload.get("submission_channel") in ["email",
"physical_via_email"]:
            if not payload.get("recipient_email"):
                payload["recipient_email"] = "lmcpaqsystem@gmail.com"

        return payload

    @classmethod
    def _ensure_quote_number(cls, payload: Dict[str, Any]) -> Dict[str,
Any]:
        result = deepcopy(payload)
        existing_quote_number = cls._safe_str(result.get("quote_number"))
        if existing_quote_number:
            return result

        buyer_rfq_number = cls._safe_str(
            result.get("_locked_buyer_rfq_number")
            or result.get("buyer_rfq_number")
            or result.get("rfq_number")
            or result.get("tender_number")
            or result.get("reference_number")
            or result.get("document_number")
        )
        if buyer_rfq_number and not
cls._is_unknown_token(buyer_rfq_number):
            result["quote_number"] = f"LMCP-{buyer_rfq_number}"
        else:
            result["quote_number"] = f"LMCP-
{datetime.now().strftime('%Y%m%d%H%M%S')}"
        return result

    @classmethod
```

```
    def _get_locked_buyer_rfq_number(cls, payload: Dict[str, Any]) ->
str:
        if not isinstance(payload, dict):
            return ""

        original_input = payload.get("_original_input_payload") or {}
        if not isinstance(original_input, dict):
            original_input = {}

        candidate = (
            payload.get("_locked_buyer_rfq_number")
            or payload.get("buyer_rfq_number")
            or payload.get("rfq_number")
            or payload.get("reference_number")
            or payload.get("document_number")
            or original_input.get("_locked_buyer_rfq_number")
            or original_input.get("buyer_rfq_number")
            or original_input.get("rfq_number")
            or original_input.get("reference_number")
            or original_input.get("document_number")
        )

        return str(candidate).strip() if candidate else ""

    def _get_locked_buyer_rfq_number(cls, payload: Dict[str, Any]) ->
str:
        if not isinstance(payload, dict):
            return ""

        original_input = payload.get("_original_input_payload") or {}
        if not isinstance(original_input, dict):
            original_input = {}

        candidate = (
            payload.get("_locked_buyer_rfq_number")
            or payload.get("buyer_rfq_number")
            or payload.get("rfq_number")
            or payload.get("reference_number")
            or payload.get("document_number")
            or original_input.get("_locked_buyer_rfq_number")
            or original_input.get("buyer_rfq_number")
            or original_input.get("rfq_number")
            or original_input.get("reference_number")
            or original_input.get("document_number")
        )

    @classmethod
    def _get_locked_buyer_rfq_number(cls, payload: Dict[str, Any]) ->
str:
        if not isinstance(payload, dict):
            return ""

        original_input = payload.get("_original_input_payload") or {}
        if not isinstance(original_input, dict):
            original_input = {}

        source_rfq = payload.get("source_rfq") or {}
        if not isinstance(source_rfq, dict):
```

```
            source_rfq = {}

        submission_pack = payload.get("submission_pack") or {}
        if not isinstance(submission_pack, dict):
            submission_pack = {}

        candidate = (
            payload.get("_locked_buyer_rfq_number")
            or payload.get("buyer_rfq_number")
            or payload.get("rfq_number")
            or payload.get("reference_number")
            or payload.get("document_number")
            or submission_pack.get("buyer_rfq_number")
            or submission_pack.get("document_number")
            or source_rfq.get("_locked_buyer_rfq_number")
            or source_rfq.get("buyer_rfq_number")
            or source_rfq.get("rfq_number")
            or source_rfq.get("reference_number")
            or source_rfq.get("document_number")
            or original_input.get("_locked_buyer_rfq_number")
            or original_input.get("buyer_rfq_number")
            or original_input.get("rfq_number")
            or original_input.get("reference_number")
            or original_input.get("document_number")
        )

        return str(candidate).strip() if candidate else ""

    @classmethod
    def _ensure_basic_identity_fields(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        result = deepcopy(payload)

        result = cls._force_rfq_into_payload(result)

        source_rfq = cls._safe_dict(result.get("source_rfq"))
        buyer = cls._safe_dict(result.get("buyer"))

        title = cls._pick_first_non_empty(
            result.get("title"),
            result.get("description"),
            source_rfq.get("title"),
            source_rfq.get("description"),
            default="Supply and delivery item",
        )
        buyer_name = cls._pick_first_non_empty(
            result.get("buyer_name"),
            buyer.get("company_name"),
            buyer.get("name"),
            source_rfq.get("buyer_name"),
            default="Client",
        )

        locked_buyer_rfq_number = cls._safe_str(
            result.get("_locked_buyer_rfq_number")
            or result.get("buyer_rfq_number")
        )
```

```
        locked_buyer_rfq_number =
cls._get_locked_buyer_rfq_number(result)

        if not locked_buyer_rfq_number:
            locked_buyer_rfq_number =
cls._get_locked_buyer_rfq_number(source_rfq)

        if not locked_buyer_rfq_number:
            locked_buyer_rfq_number = f"RFQ-
{datetime.utcnow().strftime('%Y%m%d%H%M%S')}"

        if not result.get("quote_number"):
            result["quote_number"] = f"LMCP-{locked_buyer_rfq_number}"

        result["_locked_buyer_rfq_number"] = locked_buyer_rfq_number
        result["buyer_rfq_number"] = locked_buyer_rfq_number
        result["rfq_number"] = locked_buyer_rfq_number
        result["reference_number"] = locked_buyer_rfq_number
        result["document_number"] = locked_buyer_rfq_number

        result["title"] = title
        result["buyer_name"] = buyer_name

        if not isinstance(buyer, dict):
            buyer = {}
        if not cls._safe_str(buyer.get("company_name")):
            buyer["company_name"] = buyer_name
        if not cls._safe_str(buyer.get("name")):
            buyer["name"] = buyer_name
        buyer["rfq_number"] = locked_buyer_rfq_number
        result["buyer"] = buyer

        submission_pack = cls._safe_dict(result.get("submission_pack"))
        submission_pack["buyer_rfq_number"] = locked_buyer_rfq_number
        if not cls._safe_str(submission_pack.get("document_number")) or
cls._is_unknown_token(
            submission_pack.get("document_number")
        ):
            submission_pack["document_number"] = locked_buyer_rfq_number
        result["submission_pack"] = submission_pack

        return result

    @classmethod
    def _sanitize_filename_part(cls, value: str, default: str =
"UNKNOWN") -> str:
        text = cls._safe_str(value, default)
        cleaned = "".join(ch if ch.isalnum() or ch in {"-", "_", "."}
else "-" for ch in text).strip("-._")
        return cleaned or default

        locked_buyer_rfq_number =
cls._get_locked_buyer_rfq_number(seeded_payload)
        if locked_buyer_rfq_number:
            buyer_rfq_number = locked_buyer_rfq_number

        locked_buyer_rfq_number =
cls._get_locked_buyer_rfq_number(payload)
```

```
        if locked_buyer_rfq_number:
            payload["_locked_buyer_rfq_number"] = locked_buyer_rfq_number
            payload["buyer_rfq_number"] = locked_buyer_rfq_number
            payload["rfq_number"] = locked_buyer_rfq_number
            payload["reference_number"] = locked_buyer_rfq_number
            payload["document_number"] = locked_buyer_rfq_number

        if not payload.get("quote_number") and locked_buyer_rfq_number:
            payload["quote_number"] = f"LMCP-{locked_buyer_rfq_number}"

    @classmethod
    def _ensure_quote_storage_context(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        from copy import deepcopy
        from datetime import datetime
        from pathlib import Path

        result = deepcopy(payload if isinstance(payload, dict) else {})

        locked_buyer_rfq_number =
cls._get_locked_buyer_rfq_number(result)

        if locked_buyer_rfq_number:
            result["_locked_buyer_rfq_number"] = locked_buyer_rfq_number
            result["buyer_rfq_number"] = locked_buyer_rfq_number
            result["rfq_number"] = locked_buyer_rfq_number
            result["reference_number"] = locked_buyer_rfq_number
            result["document_number"] = locked_buyer_rfq_number

        buyer_rfq_number = cls._safe_str(result.get("buyer_rfq_number"))
        quote_number = cls._safe_str(result.get("quote_number"))

        if not buyer_rfq_number:
            buyer_rfq_number = f"RFQ-
{datetime.utcnow().strftime('%Y%m%d%H%M%S')}"
            result["_locked_buyer_rfq_number"] = buyer_rfq_number
            result["buyer_rfq_number"] = buyer_rfq_number
            result["rfq_number"] = buyer_rfq_number
            result["reference_number"] = buyer_rfq_number
            result["document_number"] = buyer_rfq_number

        if not quote_number:
            quote_number = f"LMCP-{buyer_rfq_number}"
            result["quote_number"] = quote_number

        # ,úÖ THIS WAS MISSING / BROKEN
        month_folder = datetime.utcnow().strftime("%Y-%m")

        safe_buyer_rfq = cls._sanitize_filename_part(
            buyer_rfq_number if not
cls._is_unknown_token(buyer_rfq_number) else "UNKNOWN-RFQ",
            "UNKNOWN-RFQ",
        )

        safe_quote_number = cls._sanitize_filename_part(
            quote_number,
            "UNKNOWN-QUOTE",
        )
```

```
        folder_name = f"{safe_buyer_rfq}__{safe_quote_number}"

        monthly_quote_folder = Path("monthly_quotes") / month_folder /
folder_name

        result["monthly_quote_folder"] = str(monthly_quote_folder)
        result["quote_storage_dir"] = str(monthly_quote_folder)
        result["folder_name"] = folder_name
        result["buyer_rfq_number"] = buyer_rfq_number
        result["quote_number"] = quote_number

        return result

    @classmethod
    def _normalize_item_for_quote(cls, item: Dict[str, Any], idx: int) ->
Dict[str, Any]:
        quantity = cls._to_float(item.get("quantity") or item.get("qty")
or 1, 1.0)
        if quantity <= 0:
            quantity = 1.0

        unit_price = cls._to_float(item.get("unit_price") or
item.get("price") or item.get("rate") or 0, 0.0)
        line_total = cls._to_float(item.get("line_total") or
item.get("total_price") or item.get("amount") or 0, 0.0)

        if unit_price <= 0 and line_total > 0 and quantity > 0:
            unit_price = line_total / quantity
        if line_total <= 0 and unit_price > 0 and quantity > 0:
            line_total = unit_price * quantity

        return {
            "line_number": cls._safe_str(item.get("line_number") or
item.get("line_no") or idx),
            "description": cls._safe_str(
                item.get("description") or item.get("item_description")
or item.get("name") or "Supply and delivery item"
            ),
            "unit": cls._safe_str(item.get("unit") or item.get("uom") or
"Each"),
            "quantity": quantity,
            "unit_price": round(unit_price, 2),
            "line_total": round(line_total, 2),
        }

    @classmethod
    def _extract_or_build_items_for_quote(cls, payload: Dict[str, Any]) -
> List[Dict[str, Any]]:
        for key in ("items", "line_items", "buyer_pricing_schedule",
"pricing_schedule_items", "pricing_schedule"):
            candidates = payload.get(key)
            if isinstance(candidates, list) and candidates:
                items = [x for x in candidates if isinstance(x, dict)]
                if items:
                    return [cls._normalize_item_for_quote(item, idx) for
idx, item in enumerate(items, start=1)]
```

```
        description = cls._safe_str(payload.get("title") or
payload.get("description") or "Supply and delivery item")
        return [
            {
                "line_number": "1",
                "description": description,
                "unit": "Each",
                "quantity": 1.0,
                "unit_price": 0.0,
                "line_total": 0.0,
            }
        ]

    @classmethod
    def _guarantee_items_exist(cls, payload: Dict[str, Any]) -> Dict[str,
Any]:
        result = deepcopy(payload)
        items = result.get("items")
        line_items = result.get("line_items")

        candidate_items: List[Dict[str, Any]] = []

        if isinstance(items, list) and items:
            candidate_items = [deepcopy(x) for x in items if
isinstance(x, dict)]
        elif isinstance(line_items, list) and line_items:
            candidate_items = [deepcopy(x) for x in line_items if
isinstance(x, dict)]
        else:
            candidate_items =
cls._extract_or_build_items_for_quote(result)

        if not candidate_items:
            candidate_items = [
                {
                    "line_number": "1",
                    "description": cls._safe_str(
                        result.get("title") or result.get("description")
or "Supply and delivery item"
                    ),
                    "unit": "Each",
                    "quantity": 1.0,
                    "unit_price": 0.0,
                    "line_total": 0.0,
                }
            ]

        normalized_items = [
            cls._normalize_item_for_quote(item, idx)
            for idx, item in enumerate(candidate_items, start=1)
        ]

        result["items"] = deepcopy(normalized_items)
        result["line_items"] = deepcopy(normalized_items)
        return result

    @classmethod
```

```
    def _recompute_totals_from_items(cls, items: List[Dict[str, Any]]) ->
Dict[str, float]:
        subtotal = round(sum(cls._to_float(item.get("line_total"), 0.0)
for item in items if isinstance(item, dict)), 2)
        vat = round(subtotal * cls.DEFAULT_VAT_RATE, 2)
        total = round(subtotal + vat, 2)
        return {
            "subtotal_excl_vat": subtotal,
            "vat_amount": vat,
            "total_incl_vat": total,
        }

    @classmethod
    def _get_original_input_snapshot(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        raw = payload.get("_original_input_payload")
        if isinstance(raw, dict) and raw:
            return deepcopy(raw)
        return deepcopy(payload if isinstance(payload, dict) else {})

    @classmethod
    def _extract_identity_snapshot(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        source_rfq = cls._safe_dict(payload.get("source_rfq"))
        buyer = cls._safe_dict(payload.get("buyer"))
        items = payload.get("items")
        if not isinstance(items, list) or not items:
            items = payload.get("line_items")

        return {
            "title": cls._pick_first_non_empty(payload.get("title"),
source_rfq.get("title")),
            "description":
cls._pick_first_non_empty(payload.get("description"),
source_rfq.get("description")),
            "buyer_name": cls._pick_first_non_empty(
                payload.get("buyer_name"),
                buyer.get("company_name"),
                buyer.get("name"),
                source_rfq.get("buyer_name"),
            ),
            "buyer_rfq_number": cls._pick_first_non_empty(
                payload.get("_locked_buyer_rfq_number"),
                payload.get("buyer_rfq_number"),
                payload.get("rfq_number"),
                payload.get("tender_number"),
                payload.get("reference_number"),
                payload.get("document_number"),
                source_rfq.get("buyer_rfq_number"),
            ),
            "rfq_number": cls._pick_first_non_empty(
                payload.get("_locked_buyer_rfq_number"),
                payload.get("rfq_number"),
                payload.get("buyer_rfq_number"),
                payload.get("tender_number"),
                payload.get("reference_number"),
                payload.get("document_number"),
            ),
```

```
            "document_number": cls._pick_first_non_empty(
                payload.get("_locked_buyer_rfq_number"),
                payload.get("document_number"),
                payload.get("buyer_rfq_number"),
                payload.get("quote_number"),
            ),
            "submission_method":
cls._pick_first_non_empty(payload.get("submission_method"),
default="unknown"),
            "recipient_email": cls._pick_first_non_empty(
                payload.get("recipient_email"),
                payload.get("submission_email"),
                payload.get("buyer_email"),
                buyer.get("email"),
            ),
            "buyer_email": cls._pick_first_non_empty(
                payload.get("buyer_email"),
                payload.get("recipient_email"),
                buyer.get("email"),
            ),
            "quote_number":
cls._pick_first_non_empty(payload.get("quote_number")),
            "items": deepcopy(items) if isinstance(items, list) else [],
            "buyer": deepcopy(buyer),
        }

    @classmethod
    def _force_restore_original_payload(
        cls,
        target: Dict[str, Any],
        original_snapshot: Dict[str, Any],
    ) -> Dict[str, Any]:
        result = deepcopy(target)
        orig = deepcopy(original_snapshot or {})

        original_locked_rfq = cls._pick_first_non_empty(
            orig.get("buyer_rfq_number"),
            orig.get("rfq_number"),
            orig.get("document_number"),
        )
        if cls._is_meaningful_text(original_locked_rfq):
            result["_locked_buyer_rfq_number"] = original_locked_rfq
            result["buyer_rfq_number"] = original_locked_rfq
            result["rfq_number"] = original_locked_rfq
            result["reference_number"] = original_locked_rfq
            result["document_number"] = original_locked_rfq

        if cls._is_meaningful_text(orig.get("buyer_name")):
            result["buyer_name"] = orig["buyer_name"]
            result.setdefault("buyer", {})
            if not isinstance(result["buyer"], dict):
                result["buyer"] = {}
            result["buyer"]["name"] = orig["buyer_name"]
            result["buyer"]["company_name"] = orig["buyer_name"]

        if cls._is_meaningful_text(orig.get("submission_method")):
            result["submission_method"] = orig["submission_method"]
```

```
        if cls._is_meaningful_text(orig.get("recipient_email")):
            result["recipient_email"] = orig["recipient_email"]

        if cls._is_meaningful_text(orig.get("buyer_email")):
            result["buyer_email"] = orig["buyer_email"]
            result.setdefault("buyer", {})
            if not isinstance(result["buyer"], dict):
                result["buyer"] = {}
            result["buyer"]["email"] = orig["buyer_email"]

        if cls._is_meaningful_text(orig.get("title")):
            result["title"] = orig["title"]

        if cls._is_meaningful_text(orig.get("quote_number")):
            result["quote_number"] = orig["quote_number"]

        original_items = orig.get("items") or []
        if isinstance(original_items, list) and original_items:
            restored_items: List[Dict[str, Any]] = []
            subtotal = 0.0

            for idx, item in enumerate(original_items, start=1):
                if not isinstance(item, dict):
                    continue
                normalized = cls._normalize_item_for_quote(item, idx)
                restored_items.append(normalized)
                subtotal += cls._to_float(normalized.get("line_total"),
0.0)

            if restored_items:
                vat = round(subtotal * cls.DEFAULT_VAT_RATE, 2)
                total = round(subtotal + vat, 2)

                result["items"] = restored_items
                result["line_items"] = deepcopy(restored_items)

                result["subtotal"] = round(subtotal, 2)
                result["vat_amount"] = vat
                result["grand_total"] = total
                result["quotation_total"] = total

                totals = cls._safe_dict(result.get("totals"))
                totals["subtotal_excl_vat"] = round(subtotal, 2)
                totals["vat_amount"] = vat
                totals["total_incl_vat"] = total
                result["totals"] = totals

        submission_pack = cls._safe_dict(result.get("submission_pack"))
        if cls._is_meaningful_text(original_locked_rfq):
            submission_pack["buyer_rfq_number"] = original_locked_rfq
            submission_pack["document_number"] = original_locked_rfq
        if cls._is_meaningful_text(orig.get("submission_method")):
            submission_pack["submission_method"] =
orig["submission_method"]
        if cls._is_meaningful_text(orig.get("recipient_email")):
            submission_pack["recipient_email"] = orig["recipient_email"]
        if cls._is_meaningful_text(orig.get("quote_number")):
            submission_pack["quote_number"] = orig["quote_number"]
```

```
        result["submission_pack"] = submission_pack

        result = cls._ensure_basic_identity_fields(result)
        result = cls._force_rfq_into_payload(result)
        result = cls._guarantee_items_exist(result)
        return result

    @classmethod
    def _apply_test_pricing_if_needed(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        result = deepcopy(payload)
        items = result.get("items") or []
        if not isinstance(items, list) or not items:
            return result

        force_pipeline = cls._to_bool(
            result.get("force_quote_ready") or
result.get("force_pipeline") or result.get("pipeline_test_mode"),
            False,
        )
        if not force_pipeline:
            result["totals"] = cls._recompute_totals_from_items(items)
            result["subtotal"] = result["totals"]["subtotal_excl_vat"]
            result["vat_amount"] = result["totals"]["vat_amount"]
            result["grand_total"] = result["totals"]["total_incl_vat"]
            result["quotation_total"] =
result["totals"]["total_incl_vat"]
            return result

        priced_items: List[Dict[str, Any]] = []
        needs_test_pricing = False

        for idx, item in enumerate(items, start=1):
            if not isinstance(item, dict):
                continue
            normalized = cls._normalize_item_for_quote(item, idx)
            if cls._to_float(normalized.get("unit_price"), 0.0) <= 0 or
cls._to_float(normalized.get("line_total"), 0.0) <= 0:
                needs_test_pricing = True
            priced_items.append(normalized)

        if not needs_test_pricing:
            result["items"] = priced_items
            result["line_items"] = deepcopy(priced_items)
            result["totals"] =
cls._recompute_totals_from_items(priced_items)
            result["subtotal"] = result["totals"]["subtotal_excl_vat"]
            result["vat_amount"] = result["totals"]["vat_amount"]
            result["grand_total"] = result["totals"]["total_incl_vat"]
            result["quotation_total"] =
result["totals"]["total_incl_vat"]
            return result

        subtotal = 0.0
        final_items: List[Dict[str, Any]] = []
        for idx, item in enumerate(priced_items, start=1):
            normalized = cls._normalize_item_for_quote(item, idx)
            quantity = cls._to_float(normalized.get("quantity"), 1.0)
```

```
            unit_price = cls._to_float(normalized.get("unit_price"), 0.0)
            if unit_price <= 0:
                unit_price = cls.DEFAULT_TEST_UNIT_PRICE
            line_total = quantity * unit_price
            normalized["quantity"] = quantity
            normalized["unit_price"] = round(unit_price, 2)
            normalized["line_total"] = round(line_total, 2)
            subtotal += line_total
            final_items.append(normalized)

        vat = round(subtotal * cls.DEFAULT_VAT_RATE, 2)
        total = round(subtotal + vat, 2)

        result["items"] = final_items
        result["line_items"] = deepcopy(final_items)
        result["totals"] = {
            "subtotal_excl_vat": round(subtotal, 2),
            "vat_amount": vat,
            "total_incl_vat": total,
        }
        result["subtotal"] = round(subtotal, 2)
        result["vat_amount"] = vat
        result["grand_total"] = total
        result["quotation_total"] = total
        result["test_pricing_applied"] = True
        return result

    @classmethod
    def _enforce_quote_ready_if_requested(cls, payload: Dict[str, Any]) -
> Dict[str, Any]:
        result = deepcopy(payload)
        force_pipeline = cls._to_bool(
            result.get("force_quote_ready")
            or result.get("force_pipeline")
            or result.get("pipeline_test_mode"),
            False,
        )
        if force_pipeline:
            result["eligible"] = True
            result["quote_ready"] = True
            reasons = result.get("classification_reasons")
            if not isinstance(reasons, list) or not reasons:
                result["classification_reasons"] = ["Forced quote-ready
for pipeline testing"]
        return result

    @classmethod
    def _prepare_validation_safe_quote_payload(cls, payload: Dict[str,
Any]) -> Dict[str, Any]:
        result = deepcopy(payload)
        result = cls._lock_original_payload(result)
        result = cls._force_rfq_into_payload(result)
        result = cls._ensure_quote_number(result)
        result = cls._ensure_basic_identity_fields(result)
        result = cls._ensure_quote_storage_context(result)
        result = cls._guarantee_items_exist(result)

        items = cls._extract_or_build_items_for_quote(result)
```

```
        result["items"] = items
        result["line_items"] = deepcopy(items)

        totals = cls._recompute_totals_from_items(items)
        result["totals"] = totals
        result["subtotal"] = totals["subtotal_excl_vat"]
        result["vat_amount"] = totals["vat_amount"]
        result["grand_total"] = totals["total_incl_vat"]
        result["quotation_total"] = totals["total_incl_vat"]

        result = cls._apply_test_pricing_if_needed(result)
        result = cls._enforce_quote_ready_if_requested(result)

        result["submission_attachments"] = cls._dedupe_string_list(
            result.get("submission_attachments") if
isinstance(result.get("submission_attachments"), list) else []
        )
        result["supporting_documents"] = cls._dedupe_string_list(
            result.get("supporting_documents") if
isinstance(result.get("supporting_documents"), list) else []
        )
        return result

    @classmethod
    def _attach_completed_buyer_schedule(cls, result: Dict[str, Any]) ->
Dict[str, Any]:
        payload = deepcopy(result)
        submission_pack = cls._safe_dict(payload.get("submission_pack"))
        submission_pack["buyer_schedule_completed"] = True
        submission_pack["buyer_schedule_status"] = "prepared"
        payload["submission_pack"] = submission_pack
        payload["buyer_schedule_attached"] = True
        payload["buyer_schedule_status"] = "prepared"
        return payload

    @classmethod
    def _safe_attempt_csd_refresh(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        result = {
            "attempted": False,
            "success": False,
            "used_fallback": False,
            "error": "",
            "final_result": None,
        }

        try:
            auto_refresh_enabled =
cls._to_bool(payload.get("auto_refresh_csd"), False)
            pipeline_test_mode =
cls._to_bool(payload.get("pipeline_test_mode"), False)

            if pipeline_test_mode:
                result["error"] = "Auto CSD refresh skipped in pipeline
test mode."
                return result

            if not auto_refresh_enabled:
```

```
                result["error"] = "Auto CSD refresh disabled in payload."
                return result

            if cls._should_skip_external_calls(payload):
                result["error"] = "External calls skipped by payload
flag."
                return result

            refresh_result = CSDRefreshService.refresh_csd_with_fallback(
                month=None,
                year=None,
            )

            result["attempted"] = True
            result["success"] =
cls._safe_str(refresh_result.get("status")).lower() == "success"
            result["used_fallback"] =
bool(refresh_result.get("used_fallback", False))
            result["final_result"] = refresh_result
            return result

        except Exception as exc:
            result["attempted"] = True
            result["success"] = False
            result["error"] = str(exc)
            return result

    @classmethod
    def _write_minimal_pdf(cls, pdf_path: str, lines: List[str]) -> None:
        safe_lines: List[str] = []
        for line in lines:
            text = cls._safe_str(line)
            text = text.replace("\\", "\\\\").replace("(",
"\\(").replace(")", "\\)")
            safe_lines.append(text)

        content_lines = ["BT", "/F1 12 Tf", "50 780 Td"]
        first = True
        for line in safe_lines:
            if first:
                content_lines.append(f"({line}) Tj")
                first = False
            else:
                content_lines.append("0 -18 Td")
                content_lines.append(f"({line}) Tj")
        content_lines.append("ET")

        stream = "\n".join(content_lines).encode("latin-1",
errors="replace")

        objects: List[bytes] = []
        objects.append(b"<< /Type /Catalog /Pages 2 0 R >>")
        objects.append(b"<< /Type /Pages /Kids [3 0 R] /Count 1 >>")
        objects.append(
            b"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] "
            b"/Resources << /Font << /F1 4 0 R >> >> /Contents 5 0 R >>"
        )
```

```
        objects.append(b"<< /Type /Font /Subtype /Type1 /BaseFont
/Helvetica >>")
        objects.append(
            f"<< /Length {len(stream)} >>\nstream\n".encode("latin-1")
            + stream
            + b"\nendstream"
        )

        pdf = bytearray()
        pdf.extend(b"%PDF-1.4\n%\xe2\xe3\xcf\xd3\n")

        offsets = [0]
        for i, obj in enumerate(objects, start=1):
            offsets.append(len(pdf))
            pdf.extend(f"{i} 0 obj\n".encode("latin-1"))
            pdf.extend(obj)
            pdf.extend(b"\nendobj\n")

        xref_pos = len(pdf)
        pdf.extend(f"xref\n0 {len(objects)+1}\n".encode("latin-1"))
        pdf.extend(b"0000000000 65535 f \n")
        for off in offsets[1:]:
            pdf.extend(f"{off:010d} 00000 n \n".encode("latin-1"))

        pdf.extend(
            (
                f"trailer\n<< /Size {len(objects)+1} /Root 1 0 R >>\n"
                f"startxref\n{xref_pos}\n%%EOF\n"
            ).encode("latin-1")
        )

        Path(pdf_path).write_bytes(pdf)

    @classmethod
    def _build_fallback_quote_pack(cls, payload: Dict[str, Any]) ->
Dict[str, Any]:
        base_dir = Path("/app") if Path("/app").exists() else Path(".")
        output_dir = str(base_dir / "output" / "quotes")

        quote_number = cls._safe_str(payload.get("quote_number"))
        buyer_rfq_number =
cls._safe_str(payload.get("_locked_buyer_rfq_number") or
payload.get("buyer_rfq_number"))
        document_number = cls._safe_str(payload.get("document_number"),
buyer_rfq_number or quote_number or "LMCP-QUOTE")

        pdf_filename = f"{quote_number or document_number or 'LMCP-
QUOTE'}.pdf"
        pdf_path = str(Path(output_dir) / pdf_filename)

        quote_pack_result: Dict[str, Any] = {
            "quote_generated": True,
            "pdf_generated": False,
            "quote_ready": bool(payload.get("quote_ready", True)),
            "quote_number": quote_number,
            "buyer_rfq_number": buyer_rfq_number,
            "document_number": document_number,
            "pdf_path": None,
```

```
            "used_buyer_format": bool(payload.get("used_buyer_format",
False)),
            "pricing_schedule_source":
payload.get("pricing_schedule_source", "none"),
            "pricing_source": payload.get("pricing_source", "none"),
            "supplier_pricing_used":
bool(payload.get("supplier_pricing_used", False)),
            "selected_supplier_name":
payload.get("selected_supplier_name"),
            "supporting_documents": payload.get("supporting_documents",
[]),
            "submission_attachments":
payload.get("submission_attachments", []),
            "latest_csd_report": payload.get("latest_csd_report"),
            "csd_report_attached":
bool(payload.get("csd_report_attached", False)),
            "errors": [],
        }

        try:
            Path(output_dir).mkdir(parents=True, exist_ok=True)

            buyer_name_display = cls._safe_str(payload.get("buyer_name"),
"Buyer / Issuing Entity")
            title_display = cls._safe_str(payload.get("title"), "Supply
and delivery item")

            pdf_lines = [
                "LECHESA MANABA CONSULTING AND PROJECTS (PTY) LTD",
                "QUOTATION",
                "",
                f"Quote Number: {quote_number}",
                f"Buyer RFQ Number: {buyer_rfq_number or 'N/A'}",
                f"Document Number: {document_number}",
                f"Buyer / Issuing Entity: {buyer_name_display}",
                f"Quotation Title: {title_display}",
                "",
                "Prepared by Lechesa Manaba Consulting and Projects (Pty)
Ltd",
            ]

            items_for_pdf = payload.get("items") or []
            if isinstance(items_for_pdf, list) and items_for_pdf:
                pdf_lines.append("")
                pdf_lines.append("Quoted Items:")
                for idx, item in enumerate(items_for_pdf, start=1):
                    if not isinstance(item, dict):
                        continue
                    desc = cls._safe_str(item.get("description"), f"Item
{idx}")
                    qty = cls._to_float(item.get("quantity"), 1.0)
                    unit = cls._safe_str(item.get("unit"), "Each")
                    unit_price = cls._to_float(item.get("unit_price"),
0.0)
                    line_total = cls._to_float(item.get("line_total"),
0.0)
                    pdf_lines.append(
```

```
                        f"{idx}. {desc} | Qty: {qty:,.2f} {unit} | Unit:
R {unit_price:,.2f} | Total: R {line_total:,.2f}"
                    )

            cls._write_minimal_pdf(pdf_path, pdf_lines)
            quote_pack_result["pdf_generated"] = True
            quote_pack_result["pdf_path"] = pdf_path
            quote_pack_result["submission_attachments"] =
cls._dedupe_string_list([pdf_path])
            return quote_pack_result

        except Exception as pdf_exc:
            logger.exception("Fallback PDF file generation failed.")
            quote_pack_result["errors"].append(str(pdf_exc))
            quote_pack_result["quote_pack_error"] = str(pdf_exc)
            return quote_pack_result

    @classmethod
    def _build_and_attach_quote_pack(cls, result: Dict[str, Any]) ->
Dict[str, Any]:
        payload = deepcopy(result)
        submission_pack = cls._safe_dict(payload.get("submission_pack"))

        original_input_payload =
cls._get_original_input_snapshot(payload)
        original_snapshot =
cls._extract_identity_snapshot(original_input_payload)

        try:
            cls._log_stage("BEFORE_QUOTE_ENGINE", payload)

            quote_data = deepcopy(payload)
            quote_data = cls._lock_original_payload(quote_data)
            quote_data = cls._force_rfq_into_payload(quote_data)
            quote_data = cls._ensure_basic_identity_fields(quote_data)
            quote_data = cls._ensure_quote_number(quote_data)
            quote_data = cls._guarantee_items_exist(quote_data)
            quote_data =
cls._enforce_quote_ready_if_requested(quote_data)
            quote_data["_original_input_payload"] =
deepcopy(original_input_payload)

            built = build_quote_from_rfq(
                rfq=quote_data,
                company_profile=_get_default_company_profile(),
                margin_rate=str(payload.get("margin_rate") or
cls.DEFAULT_MARGIN_FLOOR),
                vat_rate="0.15",
                validity_days=30,
                delivery_days=14,
            )

            if not isinstance(built, dict):
                built = {}

            built = cls._lock_original_payload(built)
            built["_original_input_payload"] =
deepcopy(original_input_payload)
```

```
            built = cls._force_restore_original_payload(built,
original_snapshot)
            built = cls._force_rfq_into_payload(built)
            built = cls._guarantee_items_exist(built)
            built = cls._enforce_quote_ready_if_requested(built)

            safe_payload =
cls._prepare_validation_safe_quote_payload(deepcopy(built))
            safe_payload["_original_input_payload"] =
deepcopy(original_input_payload)
            safe_payload =
cls._force_restore_original_payload(safe_payload, original_snapshot)

            safe_payload =
cls._prepare_validation_safe_quote_payload(deepcopy(safe_payload))
            safe_payload["_original_input_payload"] =
deepcopy(original_input_payload)
            safe_payload =
cls._force_restore_original_payload(safe_payload, original_snapshot)

            passthrough_fields = [
                "selected_supplier_quote",
                "selected_supplier",
                "selected_supplier_name",
                "supplier_quote_files",
                "quote_comparison_json",
                "pricing_source",
                "pricing_schedule_source",
                "_locked_buyer_rfq_number",
                "buyer_rfq_number",
                "rfq_number",
                "reference_number",
                "quote_number",
                "buyer_name",
                "title",
                "document_number",
                "submission_method",
                "recipient_email",
                "buyer_email",
                "buyer",
                "seller",
                "company",
                "buyer_pricing_schedule",
                "pricing_schedule_mapped",
                "monthly_quote_folder",
                "quote_folder",
                "supplier_quotes_folder",
                "quote_pack_dir",
                "supporting_documents",
                "submission_attachments",
                "force_quote_ready",
                "force_pipeline",
                "pipeline_test_mode",
                "skip_external_calls",
                "skip_supplier_ingestion",
                "skip_email_submission",
                "auto_refresh_csd",
            ]
```

```
            for field in passthrough_fields:
                if payload.get(field) is not None:
                    safe_payload[field] = deepcopy(payload.get(field))

            safe_payload["auto_refresh_csd"] =
payload.get("auto_refresh_csd", False)
            safe_payload["pipeline_test_mode"] =
payload.get("pipeline_test_mode", False)
            safe_payload["skip_external_calls"] =
payload.get("skip_external_calls", False)
            safe_payload["skip_supplier_ingestion"] =
payload.get("skip_supplier_ingestion", False)
            safe_payload["skip_email_submission"] =
payload.get("skip_email_submission", False)
            safe_payload["_locked_buyer_rfq_number"] =
payload.get("_locked_buyer_rfq_number") or
payload.get("buyer_rfq_number")
            safe_payload["buyer_rfq_number"] =
payload.get("buyer_rfq_number")
            safe_payload["rfq_number"] = payload.get("rfq_number")
            safe_payload["reference_number"] =
payload.get("reference_number")
            safe_payload["buyer_name"] = payload.get("buyer_name")
            safe_payload["quote_number"] = payload.get("quote_number")

            safe_payload["_original_input_payload"] =
deepcopy(original_input_payload)
            safe_payload =
cls._force_restore_original_payload(safe_payload, original_snapshot)
            safe_payload = cls._force_rfq_into_payload(safe_payload)
            safe_payload =
cls._ensure_basic_identity_fields(safe_payload)
            safe_payload = cls._ensure_quote_number(safe_payload)
            safe_payload =
cls._ensure_quote_storage_context(safe_payload)
            safe_payload = cls._guarantee_items_exist(safe_payload)
            safe_payload =
cls._enforce_quote_ready_if_requested(safe_payload)

            cls._log_stage("BEFORE_CSD_REFRESH", safe_payload)
            csd_refresh_result =
cls._safe_attempt_csd_refresh(safe_payload)
            safe_payload["csd_refresh_attempted"] =
csd_refresh_result.get("attempted", False)
            safe_payload["csd_refresh_success"] =
csd_refresh_result.get("success", False)
            safe_payload["csd_refresh_used_fallback"] =
csd_refresh_result.get("used_fallback", False)
            safe_payload["csd_refresh_error"] =
csd_refresh_result.get("error", "")
            safe_payload["csd_refresh_result"] =
csd_refresh_result.get("final_result")

            cls._log_stage("BEFORE_QUOTE_PACK_SERVICE", safe_payload)
            try:
                quote_pack_result = generate_quote_pack(safe_payload)
            except Exception as quote_pack_exc:
```

```
                logger.exception("Primary quote_pack_service generation
failed, using fallback PDF builder.")
                fallback_seed = deepcopy(safe_payload)
                fallback_seed["quote_pack_error"] = str(quote_pack_exc)
                quote_pack_result =
cls._build_fallback_quote_pack(fallback_seed)

            if not isinstance(quote_pack_result, dict):
                quote_pack_result = {}

            payload["quote_generated"] =
bool(quote_pack_result.get("quote_generated", False))
            payload["pdf_generated"] =
bool(quote_pack_result.get("pdf_generated", False))
            payload["quote_ready"] =
bool(quote_pack_result.get("quote_ready", payload.get("quote_ready",
True)))
            payload["pdf_path"] = quote_pack_result.get("pdf_path")
            payload["final_pdf_path"] = quote_pack_result.get("pdf_path")
            payload["quote_pack_metadata_path"] =
quote_pack_result.get("quote_pack_metadata_path")
            payload["quote_pack_result"] = quote_pack_result
            payload["pdf_result"] = quote_pack_result
            payload["quote_data"] = safe_payload
            payload["used_buyer_format"] =
quote_pack_result.get("used_buyer_format", False)
            payload["pricing_schedule_source"] =
quote_pack_result.get("pricing_schedule_source", "none")
            payload["pricing_source"] =
quote_pack_result.get("pricing_source", "none")
            payload["supplier_pricing_used"] =
bool(quote_pack_result.get("supplier_pricing_used", False))
            payload["selected_supplier_name"] =
quote_pack_result.get("selected_supplier_name")
            payload["latest_csd_report"] =
quote_pack_result.get("latest_csd_report")
            payload["csd_report_attached"] =
bool(quote_pack_result.get("csd_report_attached", False))
            payload["supporting_documents"] = cls._dedupe_string_list(
                quote_pack_result.get("supporting_documents",
payload.get("supporting_documents", []))
            )
            payload["submission_attachments"] = cls._dedupe_string_list(
                quote_pack_result.get("submission_attachments",
payload.get("submission_attachments", []))
            )

            payload["_original_input_payload"] =
deepcopy(original_input_payload)
            payload = cls._force_restore_original_payload(payload,
original_snapshot)
            payload = cls._force_rfq_into_payload(payload)
            payload = cls._guarantee_items_exist(payload)
            payload = cls._enforce_quote_ready_if_requested(payload)

            submission_pack["quote_pack_built"] =
bool(payload.get("pdf_generated"))
```

```
            submission_pack["quote_pack_status"] = "built" if
payload.get("pdf_generated") else "failed"
            submission_pack["pdf_path"] = payload.get("pdf_path")
            submission_pack["quote_number"] = payload.get("quote_number")
            submission_pack["buyer_rfq_number"] =
payload.get("buyer_rfq_number")
            submission_pack["document_number"] =
payload.get("buyer_rfq_number")
            submission_pack["supporting_documents"] =
payload.get("supporting_documents", [])
            submission_pack["submission_attachments"] =
payload.get("submission_attachments", [])
            submission_pack["latest_csd_report"] =
payload.get("latest_csd_report")
            submission_pack["csd_report_attached"] =
payload.get("csd_report_attached", False)
            submission_pack["submission_method"] =
payload.get("submission_method")
            submission_pack["recipient_email"] =
payload.get("recipient_email")
            payload["submission_pack"] = submission_pack
            payload["quote_pack_status"] = "built" if
payload.get("pdf_generated") else "failed"

            payload = cls._ensure_basic_identity_fields(payload)
            payload = cls._ensure_quote_storage_context(payload)
            payload["_original_input_payload"] =
deepcopy(original_input_payload)
            payload = cls._force_restore_original_payload(payload,
original_snapshot)
            payload = cls._force_rfq_into_payload(payload)
            payload = cls._guarantee_items_exist(payload)
            payload = cls._enforce_quote_ready_if_requested(payload)

            cls._log_stage("AFTER_QUOTE_PACK_SERVICE", payload)
            return payload

        except Exception as exc:
            logger.exception("Quote pack generation failed.")
            payload["quote_generated"] = False
            payload["pdf_generated"] = False
            payload["quote_pack_status"] = "failed"
            payload["quote_pack_error"] = str(exc)
            submission_pack["quote_pack_built"] = False
            submission_pack["quote_pack_status"] = "failed"
            payload["submission_pack"] = submission_pack
            return payload

    @classmethod
    def _store_monthly_quote_artifacts(cls, result: Dict[str, Any]) ->
Dict[str, Any]:
        payload = deepcopy(result)
        try:
            cls._log_stage("BEFORE_MONTHLY_STORAGE", payload)

            original_input_payload =
cls._get_original_input_snapshot(payload)
```

```
            original_snapshot =
cls._extract_identity_snapshot(original_input_payload)

            payload = cls._force_restore_original_payload(payload,
original_snapshot)
            payload = cls._force_rfq_into_payload(payload)
            payload = cls._ensure_quote_storage_context(payload)
            payload = cls._guarantee_items_exist(payload)

            payload =
MonthlyQuotesStorageService.persist_pipeline_artifacts(payload)
            payload =
MonthlyQuotesStorageService.create_empty_comparison_json(payload)

            payload["_original_input_payload"] =
deepcopy(original_input_payload)
            payload = cls._force_restore_original_payload(payload,
original_snapshot)
            payload = cls._force_rfq_into_payload(payload)

            payload["monthly_quotes_status"] = "stored"
            cls._log_stage("AFTER_MONTHLY_STORAGE", payload)
            return payload
        except Exception as exc:
            logger.exception("Monthly quotes storage failed.")
            payload["monthly_quotes_status"] = "failed"
            payload["monthly_quotes_error"] = str(exc)
            return payload

    @classmethod
    def _extract_supplier_targets(cls, result: Dict[str, Any]) ->
List[Dict[str, Any]]:
        payload = deepcopy(result)
        for key in ("supplier_targets", "suppliers", "supplier_list"):
            candidates = payload.get(key)
            if isinstance(candidates, list) and candidates:
                return [x for x in candidates if isinstance(x, dict)]
        return []

    @classmethod
    def _send_supplier_rfq_requests_if_available(cls, result: Dict[str,
Any]) -> Dict[str, Any]:
        payload = deepcopy(result)

        if cls._should_skip_external_calls(payload):
            payload["supplier_rfq_requests_status"] = "skipped"
            payload["supplier_rfq_requests_message"] = "External calls
skipped by payload flag."
            payload.setdefault("supplier_rfq_requests_result", {"count":
0, "results": []})
            return payload

        supplier_targets = cls._extract_supplier_targets(payload)
        if not supplier_targets:
            payload["supplier_rfq_requests_status"] = "skipped"
            payload["supplier_rfq_requests_message"] = "No supplier
targets provided."
```

```
            payload.setdefault("supplier_rfq_requests_result", {"count":
0, "results": []})
            return payload

        buyer_rfq_number = cls._safe_str(
            payload.get("_locked_buyer_rfq_number") or
payload.get("buyer_rfq_number") or payload.get("rfq_number")
        )
        lmcp_quote_number = cls._safe_str(payload.get("quote_number"))
        buyer_name = cls._safe_str(payload.get("buyer_name"))
        buyer_title = cls._safe_str(payload.get("title") or "RFQ")

        requested_items: List[str] = []
        line_items = payload.get("line_items") or payload.get("items") or
[]
        if isinstance(line_items, list):
            for item in line_items:
                if isinstance(item, dict):
                    desc = cls._safe_str(item.get("description"))
                    if desc:
                        requested_items.append(desc)

        attachment_paths: List[str] = []
        pdf_path = cls._safe_str(payload.get("final_pdf_path") or
payload.get("pdf_path"))
        if pdf_path:
            attachment_paths.append(pdf_path)

        try:
            request_result =
SupplierRFQRequestService.send_supplier_requests(
                buyer_rfq_number=buyer_rfq_number,
                lmcp_quote_number=lmcp_quote_number,
                suppliers=supplier_targets,
                buyer_name=buyer_name,
                buyer_title=buyer_title,
                requested_items=requested_items,
                attachment_paths=attachment_paths,
            )

            payload["supplier_rfq_requests_status"] = "sent" if
request_result.get("success") else "failed"
            payload["supplier_rfq_requests_message"] =
request_result.get("message")
            payload["supplier_rfq_requests_result"] = request_result
            return payload

        except Exception as exc:
            logger.exception("Supplier RFQ request sending failed.")
            payload["supplier_rfq_requests_status"] = "failed"
            payload["supplier_rfq_requests_message"] = str(exc)
            payload["supplier_rfq_requests_result"] = {"count": 0,
"results": []}
            return payload

    @classmethod
    def _merge_supplier_ingestion_result(cls, payload: Dict[str, Any],
ingestion_result: Dict[str, Any]) -> Dict[str, Any]:
```

```
        result = deepcopy(payload)

        if not isinstance(ingestion_result, dict):
            result["supplier_quote_ingestion_status"] = "failed"
            result["supplier_quote_ingestion_error"] = "Invalid ingestion
result payload"
            return result

        defaults = {
            "supplier_quote_ingestion": ingestion_result,
            "supplier_quote_files": [],
            "supplier_quote_emails": [],
            "matched_email_ids": [],
            "supplier_quotes_found": 0,
            "supplier_quotes_saved": 0,
            "supplier_quotes_folder":
result.get("supplier_quotes_folder"),
            "quote_comparison_json": result.get("quote_comparison_json"),
            "supplier_reply_matches": [],
        }
        for key, fallback in defaults.items():
            result[key] = ingestion_result.get(key, fallback)

        for key in (
            "selected_supplier",
            "selected_supplier_name",
            "selected_supplier_quote",
            "selected_supplier_quote_path",
            "selected_quote_total",
            "supplier_quote_comparison",
            "supplier_quotes_count",
            "supplier_responses_count",
            "pricing_source",
            "supplier_pricing_used",
            "supplier_ingestion_skipped",
            "supplier_ingestion_skip_reason",
        ):
            if key in ingestion_result and ingestion_result.get(key) is
not None:
                result[key] = ingestion_result.get(key)

        result["supplier_quote_ingestion_status"] = "ingested"
        return result

    @classmethod
    def _build_supplier_ingestion_payload(cls, payload: Dict[str, Any]) -
> Dict[str, Any]:
        result = deepcopy(payload)
        result = cls._lock_original_payload(result)
        result = cls._force_rfq_into_payload(result)
        result = cls._ensure_quote_number(result)
        result = cls._ensure_basic_identity_fields(result)
        result = cls._ensure_quote_storage_context(result)
        result = cls._guarantee_items_exist(result)
        return result

    @classmethod
```

```
    def _ingest_supplier_quotes_if_present(cls, result: Dict[str, Any]) -
> Dict[str, Any]:
        payload = deepcopy(result)

        if cls._to_bool(payload.get("skip_supplier_ingestion"), False):
            payload["supplier_quote_ingestion_status"] = "skipped"
            payload["supplier_quote_ingestion"] = {
                "success": True,
                "skipped": True,
                "message": "Supplier quote ingestion skipped by payload
flag.",
            }
            payload["supplier_ingestion_skipped"] = True
            payload["supplier_ingestion_skip_reason"] =
"skip_supplier_ingestion_flag"
            return payload

        if cls._should_skip_external_calls(payload):
            payload["supplier_quote_ingestion_status"] = "skipped"
            payload["supplier_quote_ingestion"] = {
                "success": True,
                "skipped": True,
                "message": "Supplier quote ingestion skipped by
skip_external_calls flag.",
            }
            payload["supplier_ingestion_skipped"] = True
            payload["supplier_ingestion_skip_reason"] =
"skip_external_calls_flag"
            return payload

        if cls._to_bool(payload.get("pipeline_test_mode"), False):
            payload["supplier_quote_ingestion_status"] = "skipped"
            payload["supplier_quote_ingestion"] = {
                "success": True,
                "skipped": True,
                "message": "Supplier quote ingestion skipped in pipeline
test mode.",
            }
            payload["supplier_ingestion_skipped"] = True
            payload["supplier_ingestion_skip_reason"] =
"pipeline_test_mode"
            return payload

        try:
            cls._log_stage("BEFORE_SUPPLIER_INGESTION", payload)
            full_ingestion_payload =
cls._build_supplier_ingestion_payload(payload)
            ingestion_result =
ingest_supplier_quotes(full_ingestion_payload)
            merged = cls._merge_supplier_ingestion_result(payload,
ingestion_result)
            merged = cls._force_rfq_into_payload(merged)
            merged = cls._ensure_quote_storage_context(merged)
            cls._log_stage("AFTER_SUPPLIER_INGESTION", merged)
            return merged

        except Exception as exc:
            logger.exception("Supplier inbox ingestion failed.")
```

```
            payload["supplier_quote_ingestion_status"] = "failed"
            payload["supplier_quote_ingestion_error"] = str(exc)
            return payload

    @classmethod
    def _attach_supplier_inbox_quotes(
        cls,
        result: Dict[str, Any],
        supplier_ingestion_result: Optional[Dict[str, Any]] = None,
    ) -> Dict[str, Any]:
        payload = deepcopy(result)
        if supplier_ingestion_result is not None:
            payload["supplier_ingestion"] = supplier_ingestion_result
        return payload

    @classmethod
    def _submit_via_email(cls, result: Dict[str, Any]) -> Dict[str, Any]:
        payload = deepcopy(result)

        if cls._should_skip_external_calls(payload):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "flagged"
            payload["submission_message"] = "Email submission skipped
(external calls disabled)"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        if cls._should_skip_email_submission(payload):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "flagged"
            payload["submission_message"] = "Email submission skipped
(email disabled)"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        recipient_email = (
            payload.get("recipient_email")
            or payload.get("submission_email")
            or payload.get("buyer_email")
            or payload.get("buyer", {}).get("email")
            or "lmcpaqsystem@gmail.com"
        )
        payload["recipient_email"] = recipient_email

        pdf_path = cls._safe_str(payload.get("final_pdf_path") or
payload.get("pdf_path"))

        if not recipient_email:
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
```

```
            payload["submission_message"] = "No recipient email found"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        if not pdf_path:
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = "No PDF available for
submission"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        if cls._to_bool(payload.get("pipeline_test_mode"), False):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "submitted"
            payload["submission_message"] = "TEST MODE: Email submission
simulated"
            payload["email_sent"] = True
            payload["submitted_at"] = cls._now_iso()

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        try:
            from app.services.email_submission_service import
submit_quote_email

            email_result = submit_quote_email(payload=payload)
            was_sent = bool(email_result.get("success")) or
email_result.get("status") == "sent"

            payload["submission_channel"] = "email"
            payload["submission_status"] = "submitted" if was_sent else
"failed"
            payload["submission_message"] = (
                email_result.get("message")
                or email_result.get("error")
                or ("Email submission sent" if was_sent else "Email
submission failed")
            )
            payload["email_submission_result"] = email_result
            payload["email_status"] = email_result
            payload["email_sent"] = was_sent
            payload["recipient_email"] = payload.get("recipient_email")
or recipient_email or "lmcpaqsystem@gmail.com"

            if was_sent:
                payload["submitted_at"] = cls._now_iso()
```

```
            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        except Exception as exc:
            logger.exception("Email submission failed.")
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = f"Email submission failed:
{exc}"
            payload["submission_error_trace"] = traceback.format_exc()
            payload["email_sent"] = False
            payload["recipient_email"] = payload.get("recipient_email")
or recipient_email or "lmcpaqsystem@gmail.com"

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

    @classmethod
    def _submit_via_portal(cls, result: Dict[str, Any]) -> Dict[str,
Any]:
        payload = deepcopy(result)

        if cls._should_skip_external_calls(payload):
            payload["submission_method"] = "portal"
            payload["submission_channel"] = "portal"
            payload["recipient_email"] = None
            payload["submission_status"] = "flagged"
            payload["submission_message"] = "Portal submission skipped by
skip_external_calls flag"
            return payload

        payload["submission_method"] = "portal"
        payload["submission_channel"] = "portal"
        payload["recipient_email"] = None
        payload["submission_status"] = "flagged"
        payload["submission_message"] = "Portal submission engine not yet
connected"
        return payload

    @classmethod
    def _submit_via_email(cls, result: Dict[str, Any]) -> Dict[str, Any]:
        payload = deepcopy(result)

        if cls._should_skip_external_calls(payload):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "flagged"
            payload["submission_message"] = "Email submission skipped
(external calls disabled)"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"
```

```
            return payload

        if cls._should_skip_email_submission(payload):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "flagged"
            payload["submission_message"] = "Email submission skipped
(email disabled)"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        recipient_email = (
            payload.get("recipient_email")
            or payload.get("submission_email")
            or payload.get("buyer_email")
            or payload.get("buyer", {}).get("email")
            or "lmcpaqsystem@gmail.com"
        )

        pdf_path = cls._safe_str(payload.get("final_pdf_path") or
payload.get("pdf_path"))

        if not recipient_email:
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = "No recipient email found"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        if not pdf_path:
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = "No PDF available for
submission"
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        if cls._to_bool(payload.get("pipeline_test_mode"), False):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "submitted"
            payload["submission_message"] = "TEST MODE: Email submission
simulated"
            payload["email_sent"] = True
            payload["submitted_at"] = cls._now_iso()

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"
```

```
            return payload

        try:
            from app.services.email_submission_service import
submit_quote_email

            email_result = submit_quote_email(payload=payload)
            was_sent = bool(email_result.get("success")) or
email_result.get("status") == "sent"

            payload["submission_channel"] = "email"
            payload["submission_status"] = "submitted" if was_sent else
"failed"
            payload["submission_message"] = (
                email_result.get("message")
                or email_result.get("error")
                or ("Email submission sent" if was_sent else "Email
submission failed")
            )
            payload["email_submission_result"] = email_result
            payload["email_status"] = email_result
            payload["email_sent"] = was_sent

            if was_sent:
                payload["submitted_at"] = cls._now_iso()

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

        except Exception as exc:
            logger.exception("Email submission failed.")
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = f"Email submission failed:
{exc}"
            payload["submission_error_trace"] = traceback.format_exc()
            payload["email_sent"] = False

            if payload.get("submission_method") == "physical":
                payload["submission_channel"] = "physical_via_email"

            return payload

    @classmethod
    def _flag_physical_submission(cls, result: Dict[str, Any]) ->
Dict[str, Any]:
        payload = deepcopy(result)
        payload["submission_channel"] = "physical"
        payload["submission_status"] = "flagged"
        payload["submission_message"] = "Physical/courier/hand delivery
tender flagged for manual handling"
        return payload

    @classmethod
```

```
    def _flag_not_quote_ready(cls, result: Dict[str, Any]) -> Dict[str,
Any]:
        payload = deepcopy(result)
        payload["submission_channel"] = "none"
        payload["submission_status"] = "not_submitted"
        payload["submission_message"] = "RFQ not quote-ready"
        return payload

    @classmethod
    def _run_pre_quote_supplier_flow(
        cls,
        result: Dict[str, Any],
        supplier_ingestion_result: Optional[Dict[str, Any]] = None,
    ) -> Dict[str, Any]:
        payload = deepcopy(result)
        payload = cls._attach_completed_buyer_schedule(payload)
        payload = cls._lock_original_payload(payload)
        payload = cls._force_rfq_into_payload(payload)
        payload = cls._ensure_quote_number(payload)
        payload = cls._ensure_basic_identity_fields(payload)
        payload = cls._ensure_quote_storage_context(payload)
        payload = cls._guarantee_items_exist(payload)
        payload = cls._send_supplier_rfq_requests_if_available(payload)
        payload = cls._ingest_supplier_quotes_if_present(payload)

        try:
            cls._log_stage("BEFORE_SUPPLIER_AWARD_WORKFLOW", payload)
            award_result = execute_supplier_award_workflow(payload)
            if isinstance(award_result, dict):
                payload = award_result

            locked_rfq = (
                payload.get("_locked_buyer_rfq_number")
                or payload.get("buyer_rfq_number")
                or result.get("_locked_buyer_rfq_number")
                or result.get("buyer_rfq_number")
            )

            if locked_rfq:
                payload["_locked_buyer_rfq_number"] = locked_rfq
                payload["buyer_rfq_number"] = locked_rfq
                payload["rfq_number"] = locked_rfq
                payload["reference_number"] = locked_rfq
                payload["document_number"] = locked_rfq

            cls._log_stage("AFTER_SUPPLIER_AWARD_WORKFLOW", payload)
        except Exception as exc:
            logger.exception("Supplier award workflow failed.")
            payload["supplier_award_status"] = "failed"
            payload["supplier_award_error"] = str(exc)

        payload = cls._attach_supplier_inbox_quotes(payload,
supplier_ingestion_result)
        return payload

    @classmethod
    def _route_submission(
        cls,
```

```
        result: Dict[str, Any],
        supplier_ingestion_result: Optional[Dict[str, Any]] = None,
    ) -> Dict[str, Any]:
        payload = deepcopy(result)

        if payload.get("quote_ready") is not True:
            return cls._flag_not_quote_ready(payload)

        payload = cls._run_pre_quote_supplier_flow(payload,
supplier_ingestion_result=supplier_ingestion_result)
        payload = cls._build_and_attach_quote_pack(payload)

        if payload.get("pdf_generated") is not True:
            payload["submission_channel"] =
payload.get("submission_method") or "unknown"
            payload["submission_status"] = "failed"
            payload["submission_message"] = "Quote pack generation
failed"
            return payload

        payload = cls._store_monthly_quote_artifacts(payload)

        original_input_payload =
cls._get_original_input_snapshot(payload)
        original_snapshot =
cls._extract_identity_snapshot(original_input_payload)

        payload["_original_input_payload"] =
deepcopy(original_input_payload)
        payload = cls._force_restore_original_payload(payload,
original_snapshot)
        payload = cls._force_rfq_into_payload(payload)
        payload = cls._ensure_quote_storage_context(payload)
        payload = cls._guarantee_items_exist(payload)

        # ================================
        # □üîí FINAL SUBMISSION METHOD LOCK
        # ================================
        locked_submission_method = (
            payload.get("submission_method")
            or result.get("submission_method")
            or payload.get("method")
            or "email"
        )

        if isinstance(locked_submission_method, str):
            locked_submission_method =
locked_submission_method.strip().lower()
        else:
            locked_submission_method = "email"

        payload["submission_method"] = locked_submission_method

        # ================================
        # □üì¶ UPDATE SUBMISSION PACK (LOCKED)
        # ================================
        submission_pack = payload.get("submission_pack") or {}
        if not isinstance(submission_pack, dict):
```

```
            submission_pack = {}

        submission_pack["buyer_rfq_number"] =
payload.get("buyer_rfq_number")
        submission_pack["document_number"] =
payload.get("buyer_rfq_number")
        submission_pack["submission_method"] = locked_submission_method
        submission_pack["recipient_email"] = (
            payload.get("recipient_email")
            or payload.get("submission_email")
            or payload.get("buyer_email")
        )

        payload["submission_pack"] = submission_pack

        # ================================
        # □üö¶ FINAL SUBMISSION ROUTING
        # ================================
        if locked_submission_method in {"email", "e-mail", "mail"}:
            payload["submission_method"] = "email"
            payload["submission_pack"]["submission_method"] = "email"
            return cls._submit_via_email(payload)

        elif locked_submission_method in {
            "portal",
            "etender",
            "e-tender",
            "eprocurement",
            "e-procurement",
            "online",
            "website",
        }:
            payload["submission_method"] = "portal"
            payload["submission_pack"]["submission_method"] = "portal"
            return cls._submit_via_portal(payload)

        elif locked_submission_method in {
            "physical",
            "courier",
            "hand",
            "hand delivery",
            "hand-delivery",
            "manual",
        }:
            physical_redirect_email = "lmcpaqsystem@gmail.com"

            payload["submission_method"] = "physical"
            payload["recipient_email"] = physical_redirect_email
            payload["buyer_email"] = physical_redirect_email

            submission_pack = payload.get("submission_pack") or {}
            if not isinstance(submission_pack, dict):
                submission_pack = {}

            submission_pack["submission_method"] = "physical"
            submission_pack["recipient_email"] = physical_redirect_email
            submission_pack["buyer_email"] = physical_redirect_email
            payload["submission_pack"] = submission_pack
```

```
            payload["physical_submission_redirected"] = True
            payload["physical_submission_redirect_email"] =
physical_redirect_email

            return cls._submit_via_email(payload)

        else:
            payload["submission_channel"] = "unknown"
            payload["submission_status"] = "flagged"
            payload["submission_message"] = f"Unknown submission method:
{locked_submission_method}"
            return payload

        @classmethod
        def _submit_via_email(cls, result: Dict[str, Any]) -> Dict[str,
Any]:
            payload = deepcopy(result)

        if cls._should_skip_external_calls(payload):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "flagged"
            payload["submission_message"] = "Email submission skipped by
skip_external_calls flag"
            payload["email_sent"] = False

            if payload.get("physical_submission_redirected") is True:
                payload["submission_channel"] = "physical_via_email"
                payload["submission_message"] = "Physical submission
email redirect skipped by skip_external_calls flag"

            return payload

        if cls._should_skip_email_submission(payload):
            payload["submission_channel"] = "email"
            payload["submission_status"] = "flagged"
            payload["submission_message"] = "Email submission skipped by
skip_email_submission flag"
            payload["email_sent"] = False

            if payload.get("physical_submission_redirected") is True:
                payload["submission_channel"] = "physical_via_email"
                payload["submission_message"] = "Physical submission
email redirect skipped by skip_email_submission flag"

            return payload

        recipient_email = (
            payload.get("recipient_email")
            or payload.get("submission_email")
            or payload.get("buyer_email")
            or payload.get("buyer", {}).get("email")
            or "lmcpaqsystem@gmail.com"
        )
        payload["recipient_email"] = recipient_email

        pdf_path = cls._safe_str(payload.get("final_pdf_path") or
payload.get("pdf_path"))
```

```
        if not recipient_email:
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = "No recipient email found"
            payload["email_sent"] = False

            if payload.get("physical_submission_redirected") is True:
                payload["submission_channel"] = "physical_via_email"

            return payload

        if not pdf_path:
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = "No PDF quote pack available
for email submission"
            payload["email_sent"] = False

            if payload.get("physical_submission_redirected") is True:
                payload["submission_channel"] = "physical_via_email"

            return payload

        try:
            cls._log_stage("BEFORE_EMAIL_SUBMISSION", payload)
            from app.services.email_submission_service import
submit_quote_email  # type: ignore

            email_result = submit_quote_email(payload=payload)
            was_sent = bool(email_result.get("success")) or
email_result.get("status") == "sent"

            payload["submission_channel"] = "email"
            payload["submission_status"] = "submitted" if was_sent else
"failed"
            payload["submission_message"] = (
                email_result.get("message")
                or email_result.get("error")
                or ("Email submission sent" if was_sent else "Email
submission failed")
            )
            payload["email_submission_result"] = email_result
            payload["email_status"] = email_result
            payload["email_sent"] = was_sent
            payload["submitted_at"] = cls._now_iso() if was_sent else
payload.get("submitted_at")

            if payload.get("physical_submission_redirected") is True:
                payload["submission_channel"] = "physical_via_email"
                payload["submission_message"] = (
                    "Physical submission redirected by email to
lmcpaqsystem@gmail.com"
                    if was_sent
                    else "Physical submission email redirect failed"
                )

            cls._log_stage("AFTER_EMAIL_SUBMISSION", payload)
            return payload
```

```
        except Exception as exc:
            logger.exception("Email submission failed.")
            payload["submission_channel"] = "email"
            payload["submission_status"] = "failed"
            payload["submission_message"] = f"Email submission failed:
{exc}"
            payload["submission_error_trace"] = traceback.format_exc()
            payload["email_sent"] = False

            if payload.get("physical_submission_redirected") is True:
                payload["submission_channel"] = "physical_via_email"

            return payload

            if payload.get("physical_submission_redirected") is True:
                payload["submission_channel"] = "physical_via_email"
                payload["submission_message"] = (
                    "Physical submission redirected by email to
lmcpaqsystem@gmail.com"
                    if payload.get("email_sent")
                    else "Physical submission email redirect failed"
                )

    @classmethod
    def process_single_rfq(
        cls,
        rfq: Dict[str, Any],
        supplier_ingestion_result: Optional[Dict[str, Any]] = None,
    ) -> Dict[str, Any]:
        try:
            original_input_payload = deepcopy(rfq if isinstance(rfq,
dict) else {})

            original_locked_rfq = (
                original_input_payload.get("_locked_buyer_rfq_number")
                or original_input_payload.get("buyer_rfq_number")
                or original_input_payload.get("rfq_number")
                or original_input_payload.get("reference_number")
                or original_input_payload.get("document_number")
            )

            if original_locked_rfq:
                original_input_payload["_locked_buyer_rfq_number"] =
original_locked_rfq
                original_input_payload["buyer_rfq_number"] =
original_locked_rfq
                original_input_payload["rfq_number"] =
original_locked_rfq
                original_input_payload["reference_number"] =
original_locked_rfq
                original_input_payload["document_number"] =
original_locked_rfq

            cls._log_stage("START_SINGLE_RFQ", original_input_payload)

            item = cls._normalize_rfq(rfq or {})
```

```
            item["_original_input_payload"] =
deepcopy(original_input_payload)

            item = cls._force_rfq_into_payload(item)
            item = cls._guarantee_items_exist(item)

            classification = cls._classify_rfq(item)

            result = deepcopy(item)
            if isinstance(classification, dict):
                result.update(classification)

            result["_original_input_payload"] =
deepcopy(original_input_payload)

            result = cls._finalize_classification_flags(result)
            result = cls._enforce_quote_ready_if_requested(result)
            result = cls._force_rfq_into_payload(result)
            result = cls._ensure_quote_number(result)
            result = cls._ensure_basic_identity_fields(result)

            if original_locked_rfq:
                result["_locked_buyer_rfq_number"] = original_locked_rfq
                result["buyer_rfq_number"] = original_locked_rfq
                result["rfq_number"] = original_locked_rfq
                result["reference_number"] = original_locked_rfq
                result["document_number"] = original_locked_rfq

            if original_locked_rfq and not result.get("quote_number"):
                result["quote_number"] = f"LMCP-{original_locked_rfq}"

            result = cls._ensure_quote_storage_context(result)
            result = cls._guarantee_items_exist(result)

            result["pipeline_status"] = "processed"
            result["submission_status"] = result.get("submission_status")
or "not_submitted"
            result["submission_message"] = (
                result.get("submission_message")
                or "RFQ processed through LMCP tender pipeline."
            )
            result["updated_at"] = cls._now_iso()

            result = cls._route_submission(
                result,
                supplier_ingestion_result=supplier_ingestion_result,
            )

            if result.get("quote_ready") is not True:
                if result.get("force_quote_ready") or
result.get("pipeline_test_mode"):
                    result["quote_ready"] = True
                    result["pipeline_status"] = "quote_ready"
                else:
                    result["pipeline_status"] = "not_quote_ready"
            elif result.get("quote_generated") is True and
result.get("pdf_generated") is True:
                if result.get("submission_status") == "submitted":
```

```
                    result["pipeline_status"] = "submitted"
                elif result.get("submission_status") in {"flagged",
"not_submitted", "failed"}:
                    result["pipeline_status"] = "quote_built"
                else:
                    result["pipeline_status"] =
"quote_generation_complete"
            elif result.get("quote_generated") is False or
result.get("pdf_generated") is False:
                result["pipeline_status"] = "quote_build_failed"

            if result.get("submission_method") == "email":
                final_email = (
                    result.get("recipient_email")
                    or result.get("submission_email")
                    or result.get("buyer_email")
                    or result.get("buyer", {}).get("email")
                    or (result.get("submission_pack") or
{}).get("recipient_email")
                    or original_input_payload.get("recipient_email")
                    or original_input_payload.get("submission_email")
                    or original_input_payload.get("buyer_email")
                    or original_input_payload.get("buyer",
{}).get("email")
                    or "lmcpaqsystem@gmail.com"
                )
                result["recipient_email"] = final_email

                submission_pack = result.get("submission_pack") or {}
                if not isinstance(submission_pack, dict):
                    submission_pack = {}
                submission_pack["recipient_email"] = final_email
                result["submission_pack"] = submission_pack

            if original_locked_rfq:
                result["_locked_buyer_rfq_number"] = original_locked_rfq
                result["buyer_rfq_number"] = original_locked_rfq
                result["rfq_number"] = original_locked_rfq
                result["reference_number"] = original_locked_rfq
                result["document_number"] = original_locked_rfq

            result["_original_input_payload"] =
deepcopy(original_input_payload)
            result["updated_at"] = cls._now_iso()
            cls._log_stage("END_SINGLE_RFQ", result)
            return result

        except Exception as exc:
            logger.exception("Pipeline failed for RFQ item.")
            return {
                "rfq_id": (rfq or {}).get("rfq_id") or (rfq or
{}).get("id"),
                "title": (rfq or {}).get("title") or (rfq or
{}).get("name") or "Untitled RFQ",
                "eligible": False,
                "quote_ready": False,
                "quote_generated": False,
                "pdf_generated": False,
```

```
                "pipeline_status": "failed",
                "submission_status": "failed",
                "submission_message": f"Unhandled pipeline error: {exc}",
                "error_trace": traceback.format_exc(),
                "source_item": rfq,
                "updated_at": cls._now_iso(),
            }

            # ================================
            # □üîí FINAL HARD LOCK (CRITICAL)
            # ================================
            if result.get("force_quote_ready") or
result.get("pipeline_test_mode"):
                result["quote_ready"] = True
                result["eligible"] = True
                result["pipeline_status"] = result.get("pipeline_status")
or "quote_ready"

    @classmethod
    def run_tender_pipeline_batch_from_harvest(
        cls,
        items: List[Dict[str, Any]],
        source: str = "harvest",
        persist_to_live_store: bool = True,
    ) -> Dict[str, Any]:
        results: List[Dict[str, Any]] = []

        for item in items or []:
            try:
                rfq = item if isinstance(item, dict) else {}
                if source and not rfq.get("source"):
                    rfq["source"] = source
                if "persist_to_live_store" not in rfq:
                    rfq["persist_to_live_store"] = persist_to_live_store

                result = cls.process_single_rfq(rfq)
                results.append(result)
            except Exception as exc:
                logger.exception("Batch pipeline failed for RFQ item.")
                results.append(
                    {
                        "rfq_id": item.get("rfq_id") if isinstance(item,
dict) else None,
                        "title": item.get("title") if isinstance(item,
dict) else None,
                        "eligible": False,
                        "quote_ready": False,
                        "quote_generated": False,
                        "pdf_generated": False,
                        "status": "failed",
                        "error": str(exc),
                        "rfq": item if isinstance(item, dict) else None,
                        "updated_at": cls._now_iso(),
                    }
                )

        successful_quotes = sum(1 for r in results if isinstance(r, dict)
and r.get("quote_generated") is True)
```

```
        successful_pdfs = sum(1 for r in results if isinstance(r, dict)
and r.get("pdf_generated") is True)
        successful_submissions = sum(1 for r in results if isinstance(r,
dict) and r.get("submission_status") == "submitted")
        stored_monthly_quotes = sum(1 for r in results if isinstance(r,
dict) and r.get("monthly_quotes_status") == "stored")

        return {
            "status": "completed",
            "source": source,
            "persist_to_live_store": persist_to_live_store,
            "total_items": len(items or []),
            "processed_items": len(results),
            "quote_generated_count": successful_quotes,
            "pdf_generated_count": successful_pdfs,
            "submitted_count": successful_submissions,
            "monthly_quotes_stored_count": stored_monthly_quotes,
            "results": results,
            "updated_at": cls._now_iso(),
        }


def run_tender_pipeline_for_single_rfq(rfq: Dict[str, Any]) -> Dict[str,
Any]:
    return TenderPipelineService.process_single_rfq(rfq)


def run_tender_pipeline_batch_from_harvest(
    items: List[Dict[str, Any]],
    source: str = "harvest",
    persist_to_live_store: bool = True,
) -> Dict[str, Any]:
    return TenderPipelineService.run_tender_pipeline_batch_from_harvest(
        items=items,
        source=source,
        persist_to_live_store=persist_to_live_store,
    )


def run_tender_pipeline_from_payload(
    payload: Any,
    source: str = "manual",
    persist_to_live_store: bool = True,
) -> Dict[str, Any]:
    try:
        print(
            f"[PIPELINE] run_tender_pipeline_from_payload called | "
            f"payload_type={type(payload).__name__} | source={source} |
persist_to_live_store={persist_to_live_store}"
        )

        if isinstance(payload, dict):
            seeded_payload = deepcopy(payload)

            original_payload = deepcopy(seeded_payload)

            buyer_rfq_number = (
                seeded_payload.get("_locked_buyer_rfq_number")
```

```
                or seeded_payload.get("buyer_rfq_number")
                or seeded_payload.get("rfq_number")
                or seeded_payload.get("reference_number")
                or seeded_payload.get("document_number")
            )

            if not buyer_rfq_number or
str(buyer_rfq_number).strip().lower() in {
                "",
                "unknown",
                "unknown-rfq",
                "rfq-unknown",
                "n/a",
                "na",
                "-",
                "none",
                "null",
            }:
                buyer_rfq_number = f"RFQ-
{datetime.utcnow().strftime('%Y%m%d%H%M%S')}"

            seeded_payload["_locked_buyer_rfq_number"] = buyer_rfq_number
            seeded_payload["buyer_rfq_number"] = buyer_rfq_number
            seeded_payload["rfq_number"] = buyer_rfq_number
            seeded_payload["reference_number"] = buyer_rfq_number
            seeded_payload["document_number"] = buyer_rfq_number

            # ================================
            # □üîí LOCK SUBMISSION METHOD (CRITICAL)
            # ================================
            submission_method = (
                seeded_payload.get("submission_method")
                or seeded_payload.get("method")
                or "email"
            )

            seeded_payload["submission_method"] = submission_method

            if not seeded_payload.get("quote_number"):
                seeded_payload["quote_number"] = f"LMCP-
{buyer_rfq_number}"

            if source and not seeded_payload.get("source"):
                seeded_payload["source"] = source

            if "persist_to_live_store" not in seeded_payload:
                seeded_payload["persist_to_live_store"] =
persist_to_live_store

            if "_original_input_payload" not in seeded_payload:
                seeded_payload["_original_input_payload"] =
original_payload

            seeded_payload =
TenderPipelineService._lock_original_payload(seeded_payload)
            seeded_payload =
TenderPipelineService._force_rfq_into_payload(seeded_payload)
```

```
            seeded_payload =
TenderPipelineService._guarantee_items_exist(seeded_payload)
            seeded_payload =
TenderPipelineService._enforce_quote_ready_if_requested(seeded_payload)

            return
TenderPipelineService.process_single_rfq(seeded_payload)

        if isinstance(payload, list):
            return
TenderPipelineService.run_tender_pipeline_batch_from_harvest(
                items=payload,
                source=source,
                persist_to_live_store=persist_to_live_store,
            )

        return {
            "status": "failed",
            "message": "Unsupported payload type",
            "payload_type": type(payload).__name__,
            "source": source,
            "persist_to_live_store": persist_to_live_store,
        }

    except Exception as exc:
        logger.exception("run_tender_pipeline_from_payload failed.")
        return {
            "status": "failed",
            "message": f"Pipeline execution failed: {exc}",
            "error_trace": traceback.format_exc(),
            "source": source,
            "persist_to_live_store": persist_to_live_store,
        }


def _get_default_company_profile() -> Dict[str, Any]:
    return {
        "company_name": "Lechesa Manaba Consulting and Projects (Pty)
Ltd",
        "registration_number": "2012/159509/07",
        "vat_number": "4260295953",
        "email": "lechesam@me.com",
        "phone": "0826338492",
        "address": "1787 Dube Street, Batho Location, Bloemfontein,
9323",
    }
```